

Modern Computer Vision Through the Convolutional Neural Network

Harrison Green

December 2025

Abstract

Convolutional neural networks (CNNs) are one of the most influential developments in modern machine learning, enabling computers to interpret visual information with extremely high levels of accuracy—this field is called "computer vision". Their success is formed from the fusion of artificial neural network principles with the mathematical process of convolution (more specifically, cross-correlation), producing systems that efficiently and effectively learn complex spatial features. This paper provides a mathematical overview of convolutional processing, nonlinear transformations, and backpropagation in CNNs, focusing on how these components collectively empower contemporary computer vision systems.

1 Introduction

The convolutional neural network (CNN) was developed through the late 1980s and popularized by Yann LeCun in 1998, with a model designed to recognize handwritten digits. Inspired by the visual cortex of natural creatures, the mathematical principle of convolution has strong synergy when paired with the human brain-based artificial neural network. [1]

CNNs work largely the same as the standard neural networks: samples are procedurally moved through the network, changing with weights and biases. The primary distinction between the model types is the employment of the mathematical process of convolution by the CNN, allowing the model to recognize not only the individual values from the input, but also the proximity and relationship between values; this variance causes the CNN to excel at processing visual data.

Today, CNNs are the most prominent and widely used tool in the field of computer vision: the study and creation of methods for computers to see the world. This paper aims to develop an overview of this contemporary mode of computer vision from a mathematical point of view.

2 The Foundational Neural Network

2.1 Historical Background

The standard artificial neural network was formally presented by Warren McCulloch and Walter Pitts in 1943. Since then, it has served as a foundation for modern artificial intelligence, from translative tools to stock market pattern recognition. [2]

2.2 Mathematical Overview

2.3 Training of the Standard Artificial Neural Network

The training of a standard artificial neural network (ANN) can be divided into three primary processes: *forward propagation*, *loss calculation*, and *backpropagation*. Each step successively transforms input data into predicted probability of identification, optimizing model parameters for the sake of minimal error.

2.3.1 Forward Propagation

Forward propagation describes the process of processing input through the network. In other words, it is the process of digesting samples so that they are ready to be learned from.

We begin by converting input data into numerical form. For instance, a grayscale image can be represented as a matrix of pixel intensities, with each pixel value normalized to the interval $[0, 1]$:

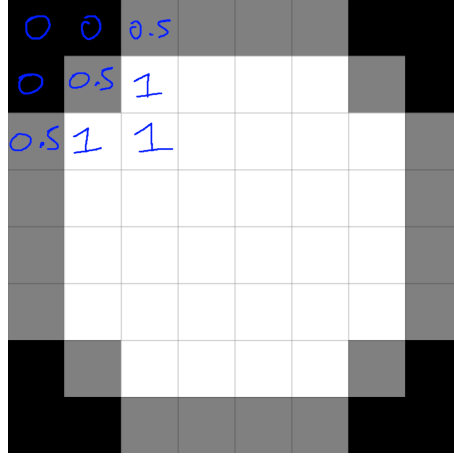


Figure 1: A possible numerical interpretation of a grayscale image.

Each of these pixel values corresponds to a neuron in the input layer. Consider, for example, a simple network designed to process four input neurons (e.g., a 2×2 image):

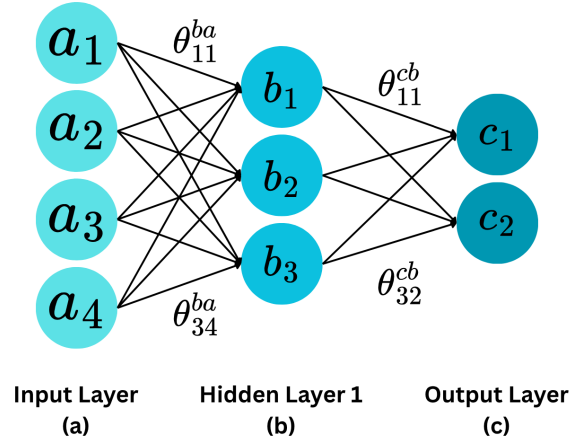


Figure 2: A simple artificial neural network demonstrating full connectivity between layers.

For a hidden layer with three neurons, the raw pre-activation values are computed as

$$\begin{bmatrix} b_1^{\text{raw}} \\ b_2^{\text{raw}} \\ b_3^{\text{raw}} \end{bmatrix} = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} & \theta_{14} \\ \theta_{21} & \theta_{22} & \theta_{23} & \theta_{24} \\ \theta_{31} & \theta_{32} & \theta_{33} & \theta_{34} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} + \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix}.$$

Here, a_m denotes the activations of the previous layer, θ_{mn} is the weight connecting neuron n in one layer to neuron m in the next, and β_m is the bias term of neuron m .

Each raw output b_m^{raw} is then passed through an activation function to produce the final activation b_m . Common activations include:

$$\text{ReLU: } b_m = \max(0, b_m^{\text{raw}}), \quad \text{Tanh: } b_m = \tanh(b_m^{\text{raw}}), \quad \text{Sigmoid: } b_m = \frac{1}{1 + e^{-b_m^{\text{raw}}}}.$$

The activation function introduces nonlinearity; without the activation function, the network would collapse to a single linear transformation, severely inhibiting learning ability.

This computation is repeated layer by layer until reaching the output neurons, whose pre-activation values (logits) represent the model's predictions.

2.3.2 Loss Calculation

When training a model, it is vital that we have a method to numerically quantify the error; this allows us to continue to use mathematics to train and tweak our model. The value we assign to represent the error of the model, per iteration, is loss.

Loss computation begins by converting the newly created logits into probability. For neural networks with multiple output classes, we apply an exponential softmax function for each logit, producing probabilities $p_k \in [0, 1]$, where $p_1 + p_2 + \dots + p_n = 1$:

$$p_k = \frac{e^{c_k}}{\sum_{j=1}^n e^{c_j}}$$

From these probabilities, we can compute the loss of the network: a value that we use to quantify the error of the ANN, for the sake of efficient training:

$$\text{Loss} = - \sum_{j=1} r_j (\log(p_j))$$

Where r refers to the true, target probability of each output (generally only either 100% or 0%). This loss model is simply known as "log loss"—taking the negative log of the predicted probability of the true outcome. The loss becomes greater with the distance p grows from 100%, exponentially punishing false confidence.

2.3.3 Backpropagation

With a loss value, backward propagation is the process of modifying weights and biases in hope that future iterations will feature better performance; training the model.

We start by calculating how much each weight contributed to the loss value we previously computed. To do this, we will employ the partial derivative of the loss, with respect to the weights between these layers. The weight connecting hidden neuron b_n to output neuron c_m is denoted as θ_{mn}^{cb} . The partial derivative is then:

$$\frac{\partial \text{Loss}}{\partial \theta_{mn}^{cb}} = \delta_m^c \cdot b_n$$

δ_m^c denotes the error of neuron c_m , and we use this formula to get the error of the weight. Since we are computing the error between our final two layers, we can apply the softmax and cross-entropy functions to simplify the equation for a simple definition of the error of neurons in the output layer:

$$\frac{\partial \text{Loss}}{\partial \theta_{mn}} = (p_m - r_m) \cdot b_n$$

In order to calculate weight error in earlier layers, we are unable to simplify the derivative with softmax and cross-entropy in the same way we could for hidden $\rightarrow$ output. So, we use a recursive definition:

$$\delta_n^b = \left(\sum_{m=1}^M \delta_m^c \cdot \theta_{mn}^{cb} \right)$$

Where M denotes the total number of neurons in layer c . The process to compute bias error is similar, albeit slightly simpler. Since biases correspond to individual neurons, we can define bias error as equal to our neuron error:

$$\frac{\partial \text{Loss}}{\partial \beta_m^c} = (p_m - r_m)$$

And between hidden layers:

$$\frac{\partial \text{Loss}}{\partial \beta_m^c} = \delta_m^c$$

With our errors computed, we can proceed in updating each parameter using the following formulas:

$$\theta := \theta - \eta \cdot \frac{\partial \text{Loss}}{\partial \theta} \quad \text{and} \quad \beta := \beta - \eta \cdot \frac{\partial \text{Loss}}{\partial \beta}$$

Here, η is the learning rate: a tunable constant that controls the size of updates. A common tuning rate is 10^{-2} (0.01), providing a strong balance between speed and stability; a model will learn quickly without oscillating around optimal weight values. This practice repeats—usually with batches of samples and averaged loss values before conducting backpropagation.

2.4 Limitations and Inefficacies

While powerful for structured, low-dimensional data, the standard ANN faces several computational and theoretical inefficiencies when applied to visual data such as high resolution images. For instance, an RGB image with size 224×224 , contains over 150,000 ($224 \cdot 224 \cdot 3$) neurons. If this were connected to even a modest hidden layer of 1,000 neurons, $150 \cdot 10^6$ weights would be created, stored, and incrementally tweaked.

Furthermore, an ANN treats each neuron is independent, leaving pattern recognition to the end of the network where layers are tighter. This can destroy local spatial relationships. For instance, two adjacent pixels are treated no differently than two distant ones. Resultingly, the ANN cannot recognize local features such as edges or corners without learning them implicitly, which is a highly inefficient and data-hungry practice.

3 Convolutional Processing

The convolutional neural network circumvents the shortcomings of its base architecture through the practice of discrete convolution, creating a system that not only better preserves accuracy, but also works significantly more efficiently.

3.1 Image Representation and Convolution

Mathematically, the process of discrete convolution is an algorithm applied to two datasets to combine them—in the convolutional neural network, we refer to these as the input feature map and the kernel (or filter). The input feature map represents the numerical form of an image, while the kernel is a smaller matrix that moves (or slides) across the image, performing a specific multiplication and summation at each position: this process is convolution.

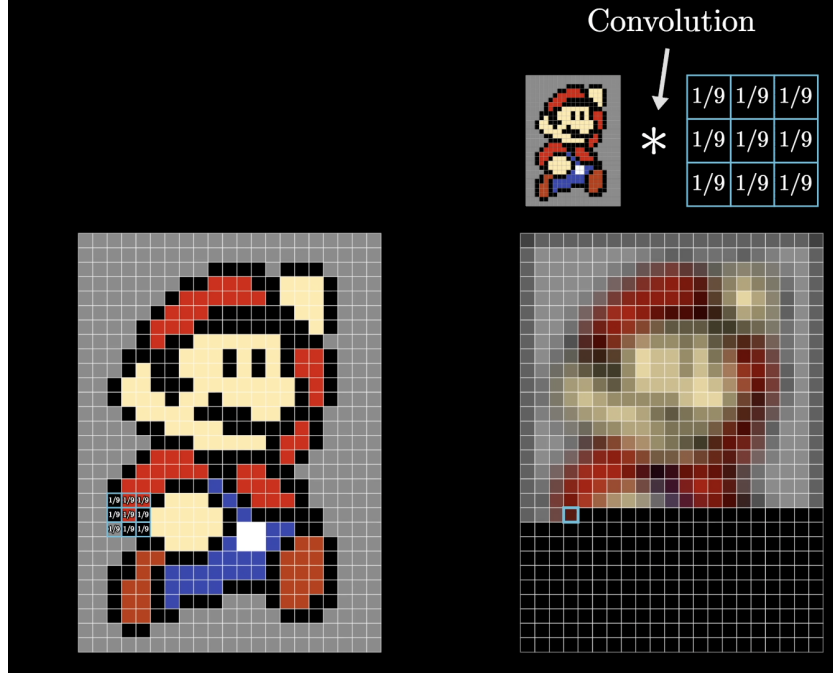


Figure 3: A 3x3 kernel of exclusively 1/9 (reciprocal of the area of the kernel, so that each pixel under the kernel is 'blended together') to create a blur effect.[3]

For an input matrix $A \in \mathbb{R}^{H \times W}$ and a kernel $B \in \mathbb{R}^{k \times k}$, the convolution for each value of matrix A (or each pixel of the input image) at $[i, j]$ is defined as:

$$(A * B)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} A[i + m, j + n] B[m, n].$$

More precisely known as cross-correlation, this convolution-adjacent process produces a new matrix, representing a filtered version of the input image. Mathematically, a kernel is a small, two-dimensional matrix of weights, where each element defines the value of a particular pixel in a $k \times k$ area. For a kernel $B \in \mathbb{R}^{k \times k}$, we can write:

$$B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}.$$

We procedurally take this smaller matrix, and "slide" it across each element of the larger matrix. In our instance, for every $A[i, j]$, we 'place' the kernel B over matrix A such that b_{11} 'overlaps' with $A[i, j]$. We then perform a multiplicative and additive algorithm to each overlapping pixel, like so:

$$C[i, j] = \sum_{m=0}^2 \sum_{n=0}^2 A[i + m, j + n] b_{mn}.$$

In this instance, each value of the kernel is a coefficient that amplifies or suppresses pixel intensities based on their spatial arrangement. During training, these coefficients are not predetermined—they begin randomly, but are optimized through backpropagation, so that the kernel learns to respond strongly to specific spatial patterns. For instance, a kernel such as

$$B_{\text{horizontal}} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

detects horizontal edges; when there is a stark difference between the brightnesses of pixels on either side of the current point, $|C[i, j]|$ will produce a very high value. Adjacentlly, the kernel

$$B_{\text{vertical}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

detects vertical edges.

Although these matrices are static examples, CNNs learn hundreds or thousands of such filters through training. Each kernel corresponds to a distinctive feature type; early layers learn to detect simple geometric features such as edges and corners, while deeper layers combine primitive patterns to recognize increasingly abstract shapes and objects.

Because each kernel is significantly smaller than the original image, the convolutional network saves time by training and updating comparatively hundreds or thousands of kernel values, as opposed to millions (often more) of weights and biases. Furthermore, by applying the same kernel across the entire image, the CNN enforces weight sharing, a technique that drastically reduces the number of variables to keep track of while ensuring the network’s detection of reoccurring features anywhere within the image. These two principles—local connectivity and weight sharing—are the foundation of the CNN’s efficiency over the standard ANN.

3.2 Pattern Detection

After initial processing, further convolutional layers continue to operate, working as feature extractors and pattern detectors. While early layers of a CNN typically learn low-level, primitive visual features such as edges, corners, and simple textures—this is because kernels with smaller fields like those present in early layers are naturally altered by simple features—as information flows deeper into the network, successive convolutions combine simpler features into more advanced visual motifs including textures with both corners and edges.[6]

Pattern detection is further enhanced by two concepts inherent to the structure of the kernel system:

- **Locality**—convolutional processes only observe a small neighborhood of the input at once, encouraging the network to detect spatially local relationships.
- **Weight sharing**—Because a kernel and its weights are applied across all spatial positions, features learned from one section of an image can be recognized and optimally processed anywhere in the image.

Together, these principles enhance a CNN’s ability to efficiently represent visual structures with far fewer parameters than a standard ANN.

3.3 Convolutional Backpropagation

While training standard neural layers requires the computation of every individual weight and bias value, convolutional backpropagation is limited to computing kernel error (how each kernel affects the loss). Because convolution is linear with respect to its weights, its gradient has a clean, structured form.

For a convolutional process on layer A with kernel B :

$$C[i, j] = \sum_{m, n} A[i + m, j + n] \cdot B[m, n],$$

the gradient of the loss with respect to the kernel B is

$$\frac{\partial \text{Loss}}{\partial B[m, n]} = \sum_{i, j} \delta_C[i, j] \cdot A[i + m, j + n],$$

where $\delta_C[i, j]$ denotes the backpropagated error at spatial position i, j .

From this expression, we know that kernel updates depend on correlations between error and localized image patches. Adjacentlly, the gradient with respect to the input feature map is computed by convolving the error map with a spatially flipped version of the kernel.[5]

4 Conclusion

Convolutional neural networks form the mathematical and conceptual backbone of modern computer vision. Their structural and mathematical advantages over traditional, fully connected networks enable them to scale to large datasets and complex tasks, while the process of convolution extracts features from images in a uniquely effective way.

As surveyed in works such as Khan et al. (2019)[7] and Zhao et al. (2024)[8], modern architectural refinements—including deeper stacks, residual connections, and multi-branch topologies—have further expanded CNN capacity while preserving its signature efficiency. As newer paradigms and technology continue to shape the field, the convolutional neural network remains fundamental to computer vision, both in concept and in computations.

References

- [1] IBM. (2021, October 6). *Convolutional Neural Networks*. *Ibm.com*. <https://www.ibm.com/think/topics/convolutional-neural-networks>
- [2] Green, H. (2025). *A mathematical overview of forward and backward propagation in artificial neural networks*. *Unpublished manuscript*. https://bingst0n.github.io/A_Mathematical_Analysis_of_Forward_and_Backward_Propagation_in_Artificial_Neural_Networks.pdf
- [3] 3Blue1Brown. (2017, September 21). *But what *is* a convolution? [Video]*. *YouTube*. <https://youtu.be/KuXjwB4LzSA>
- [4] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet classification with deep convolutional neural networks*. <https://proceedings.neurips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>
- [5] Srinivas, S., Sarvadevabhatla, R. K., Mopuri, K. R., Prabhu, N., & Babu, R. V. (2016). *A taxonomy of deep convolutional neural nets for computer vision*. <https://arxiv.org/abs/1601.06615>
- [6] Gu, J., Wang, Z., Kuen, J., Ma, L., Shahroudy, A., Shuai, B., ... & Wang, G. (2017). *Recent advances in convolutional neural networks*. <https://www.ntu.edu.sg/docs/librariesprovider106/publications/data-science-machine-learning-and-optimization/recent-advances-in-convolutional-neural-networks-2017.pdf>
- [7] Khan, A., Sohail, A., Zahoora, U., & Qureshi, A. S. (2019). *A survey of the recent architectures of deep convolutional neural networks*. <https://arxiv.org/pdf/1901.06032>
- [8] Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024). *A review of convolutional neural networks in computer vision*. <https://link.springer.com/article/10.1007/s10462-024-10721-6>